Morphology

What is Morphology ?

- Morphology is the study of the structure of words.
- Words are composed of **morphemes**
- Morphemes are the smallest meaningful units of language
- Examples:

  ✓Morpheme+s are the small+est mean+ing+ful unit+s of language.

Use cases

- Search engines: e.g. a search for 'fox' should return documents containing 'foxes', and vice versa.

- Spell checking: e.g. is `walking' a possible word form?

- …any NLP tasks will typically involve morphological parsing as a prerequisite

English Morphology

- English Language is primarily concatenative
  - stems and affixes (prefixes, suffixes, infixes and circumfixes) are concatenated to form words.

- Four concatenative processes:
  - ✓Inflection (stem + grammar affix):
    - ✓word does not change its grammatical class (walk —> walking, go—>went, fox—>foxes, book->books)
  - ✓Derivation (stem + grammar affix):
    - ✓word changes its grammatical form (computerize —> computerization, do —> doable, nation—> national,…)
  - ✓Cliticization: I've, we're, he's, …
  - ✓Compounding (stems together): doghouse, notebook, ….

Morphological Parsing

- ✓ Recognising that a word (like foxes) breaks down into component morphemes (fox and -es)
- ✓ Building a structured representation of this
  - ✤ cats —— > cat +N +PL
  - ✤ geese ——> goose +N +PL

Morphological Parsing

- Surface form: cats walking
- Lexical form: cat+N+PL walk+V+PresPart

- We should take into account of:
  - ✓Irregular forms (e.g. goose —> geese)
  - ✓Systematic rules (e.g. ‘e’ inserted before suffix ’s’ after s,x,z,ch,sh: fox —> foxes, watch —> watches)

How expressive is English morphology?

- Morphemes are concatenated in a rather "regular" way.
- Their order is well prescribed, even in morphologically rich languages.
- **Finite-state** machines are a good way to model morphology.
  - No need for "memory" to model it (no long-range dependencies).

Modeling Morphological Parsing

- Parsing here means going from the surface to the lexical form.
  - ✓ foxes —> fox +N +PL.
- Generation is the opposite process.
  - ✓ fox +N +PL —> foxes.
- It’s helpful to consider these two processes together.
- Either way, it's often useful to proceed via an intermediate form,
  - ✓Surface form: foxes
  - ✓Intermediate form: fox ^ s #
    - ✓ ^ means morpheme boundary, # means word boundary
  - ✓Lexical form: fox +N +PL

Modeling Morphological Parsing

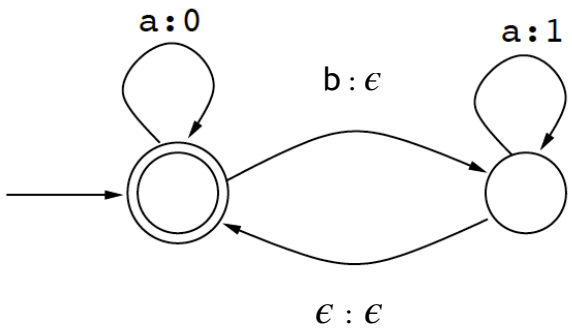
- We require one functionality
  - ✓ that maps surface forms to intermediate forms
- and another functionality
  - ✓ that maps intermediate forms to lexical forms

- How do we represent a (possibly infinite) set of strings?
- How do we represent a mapping between two (possibly infinite) sets of strings?

Finite State Transducer

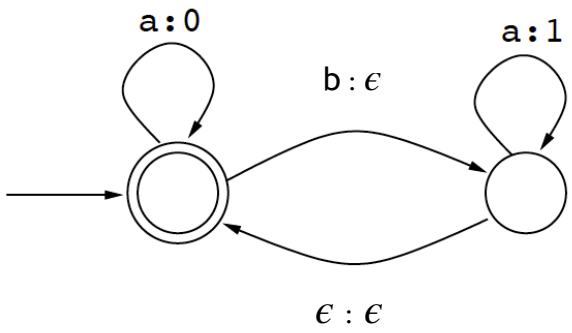
- Essentially is a finite-state automaton
  - ✓ that works on two (or more) tapes.
- They read from one of the tapes and write onto the other.
- Example of a transducer that translates as into bs

Quick Exercise



- What output will this produce, given the input aabaaabbab?

Quick Exercise



- What output will this produce, given the input aabaaabbab?
  - 001110 ??
  - 001111 ??
  - 0011101 ??
  - ……

- More than one output is possible

Finite State Transducer

A finite state transducer is a tuple $(\Sigma_i, \Sigma_o, Q, q_0, F, \Delta)$

Σ_i is the *input* alphabet

Σ_o is the *output* alphabet

Q a finite set of states

q_0 is the start state, $q_0 \in Q$

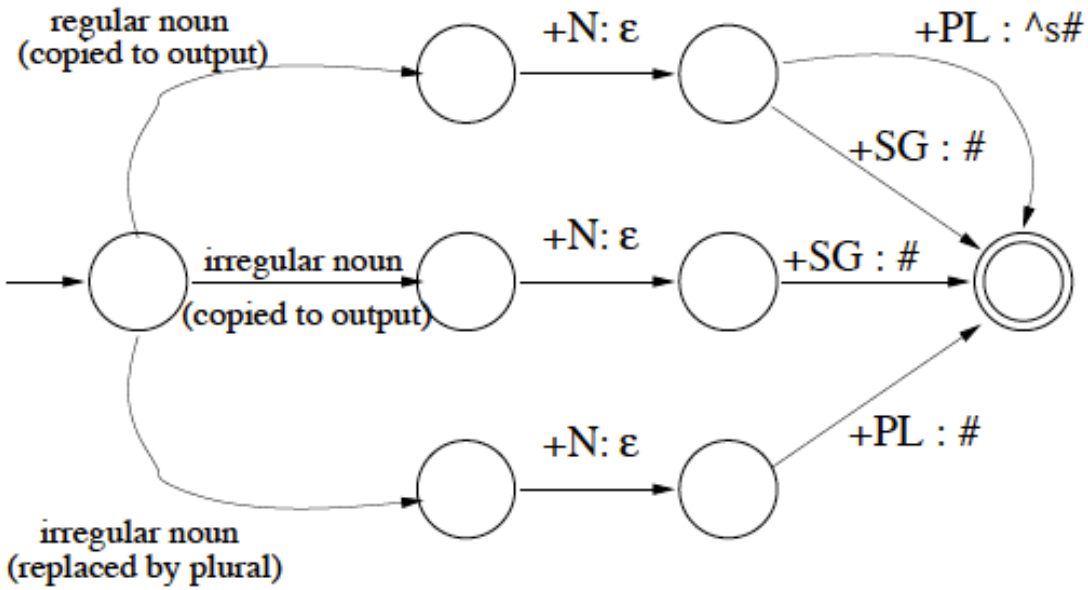
F is the set of accepting states, $F \subseteq Q$

Δ is a relation $(\Delta : Q \times \Sigma_i \rightarrow Q \times \Sigma_o)$

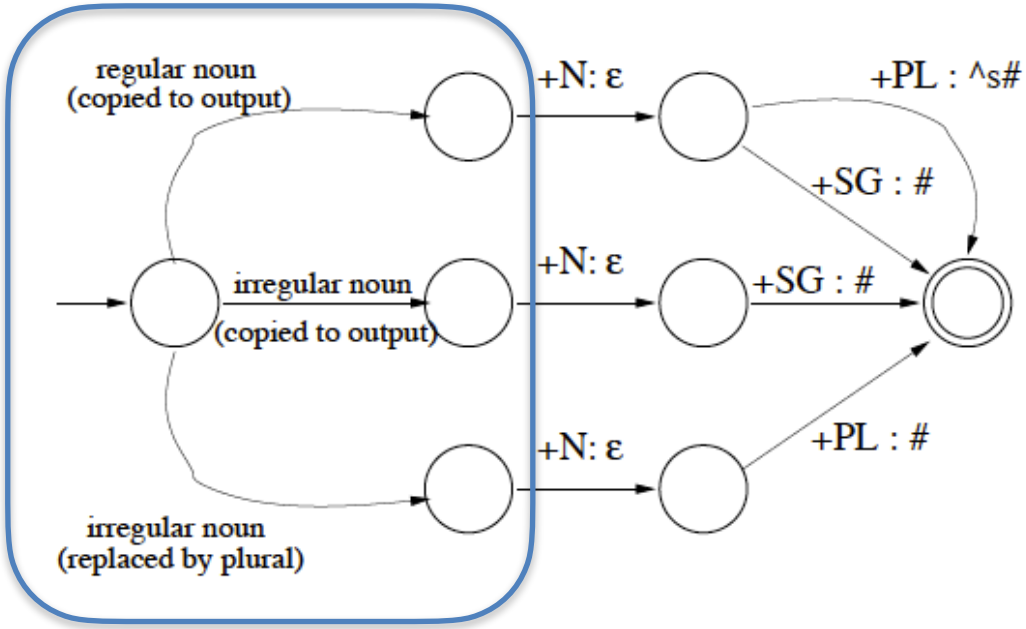
- ✓ *Note that a finite state transducer can be run in either direction!*
- ✓ *From T_1 as above, we can obtain another transducer T_2 just by swapping the roles of inputs and outputs.*

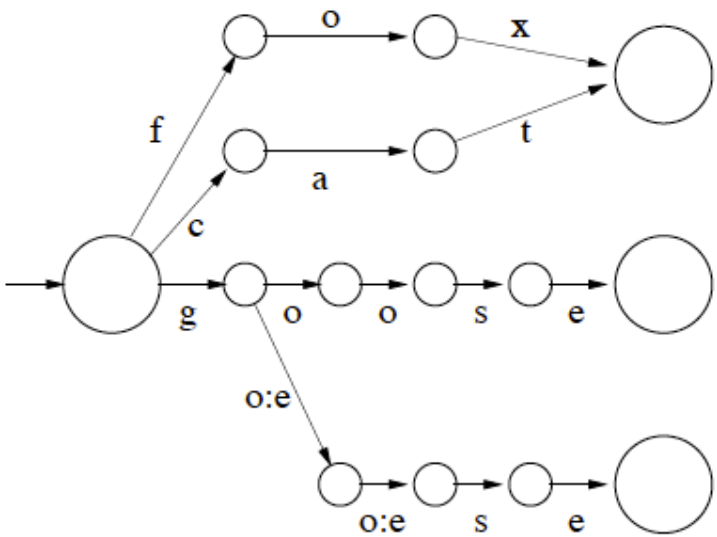
Stage 1: From lexical to intermediate form

- fox+N+PL —> fox ^ s #
- taking account of irregular forms like goose/geese.
- Treat each of +N, +SG, +PL as a single symbol.
- The transition labelled +PL : ^s# abbreviates three transitions:
  +PL : ^ ϵ : s ϵ : #



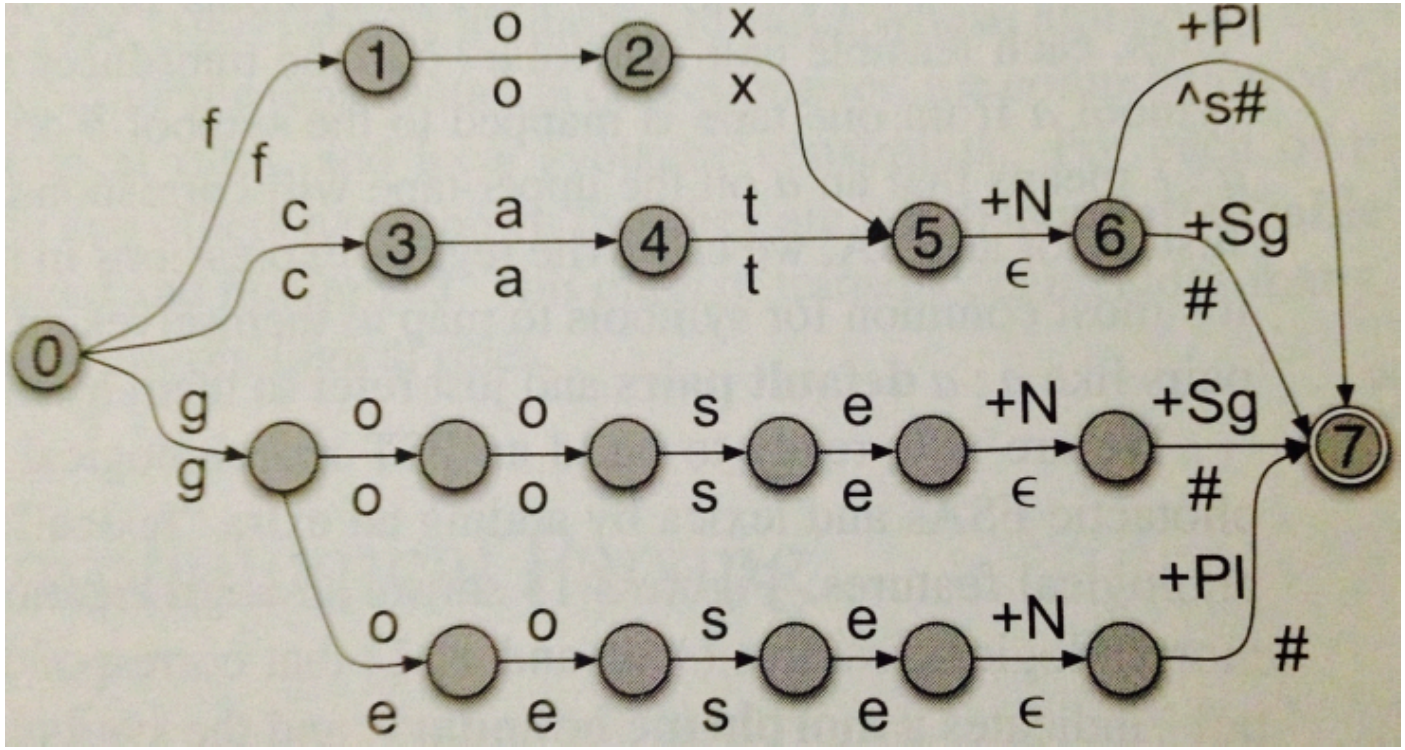
Stage 1: From lexical to intermediate form





- Here, for simplicity, a single label u abbreviates u : u.

Stage 1: From lexical to intermediate form



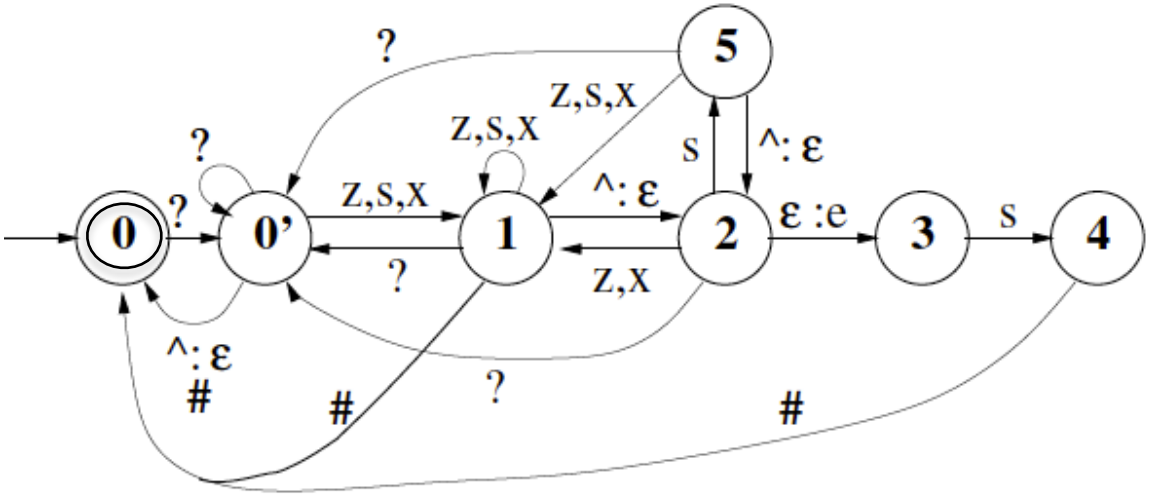
Stage 2: From intermediate to surface form

- fox ^ s # —> foxes

- To convert a sequence of morphemes to surface form, we apply a number of orthographic rules such as the following.
  - ✓E-insertion: Insert e after z,s,x before final morpheme -s. (fox —> foxes)
  - ✓E-deletion: Delete e before a suffix beginning with e,i. (love —> loving)
  - ✓Consonant doubling: Single consonants b,s,g,k,l,m,n,p,r,s,t,v are doubled before the suffix -ed or -ing. (beg —> begged)

A transducer for E-insertion

- fox ^ s # —> foxes



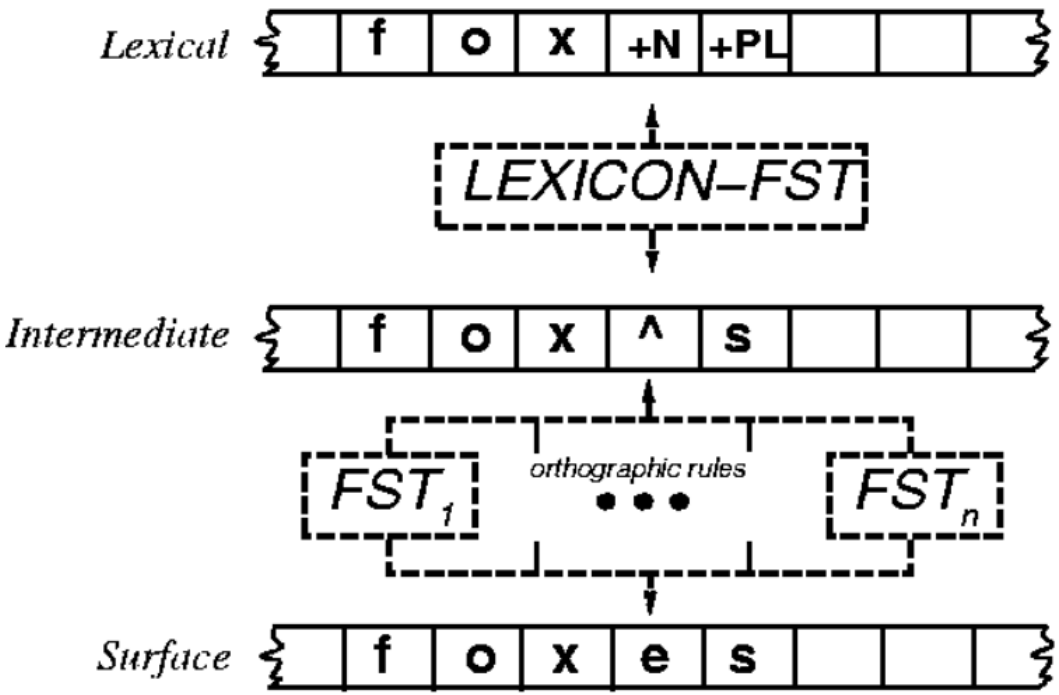
- Here ? stand for any symbol except z,s,x,^,#.
- At a morpheme boundary following z,s,x, we arrive in State 2.
- If the ensuing input sequence is s#, our only option is to go via states 3 and 4. Note that there's no # transition out of State 5.
- State 5 allows e.g. `ex^service^men#' to be translated to `exservicemen'.

Putting it all together

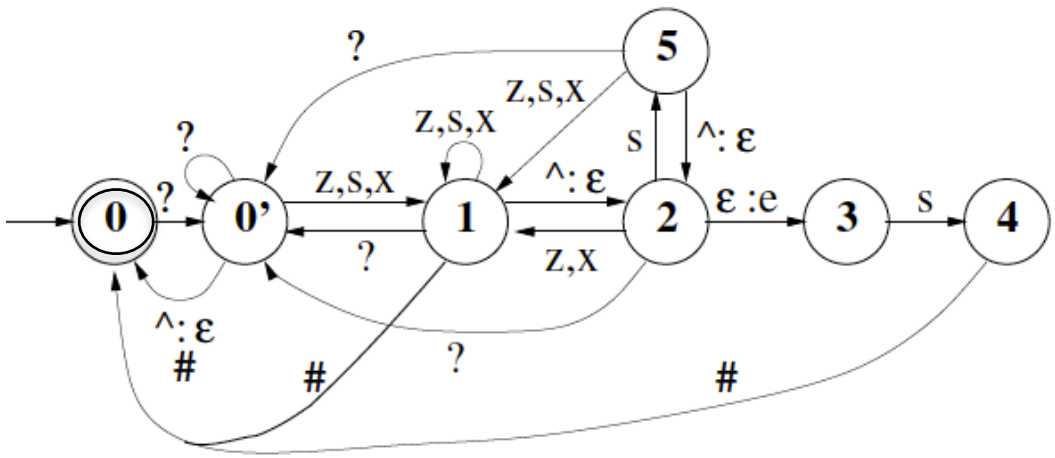
- FSTs can be composed: output from one can be input to another.
- To go from lexical to surface form, use ‘Stage 1’ transducer followed by a bunch of orthographic rule ‘Stage 2’ transducers like the above.

- The results of this generation process are typically deterministic (each lexical form gives a unique surface form)
- Running the same cascade backwards lets us do parsing (surface to lexical form).
  - Because of ambiguity, this process is frequently non-deterministic: e.g. `foxes' might be analysed as fox+N+PL or fox+V+Pres+3SG.

Putting it all together

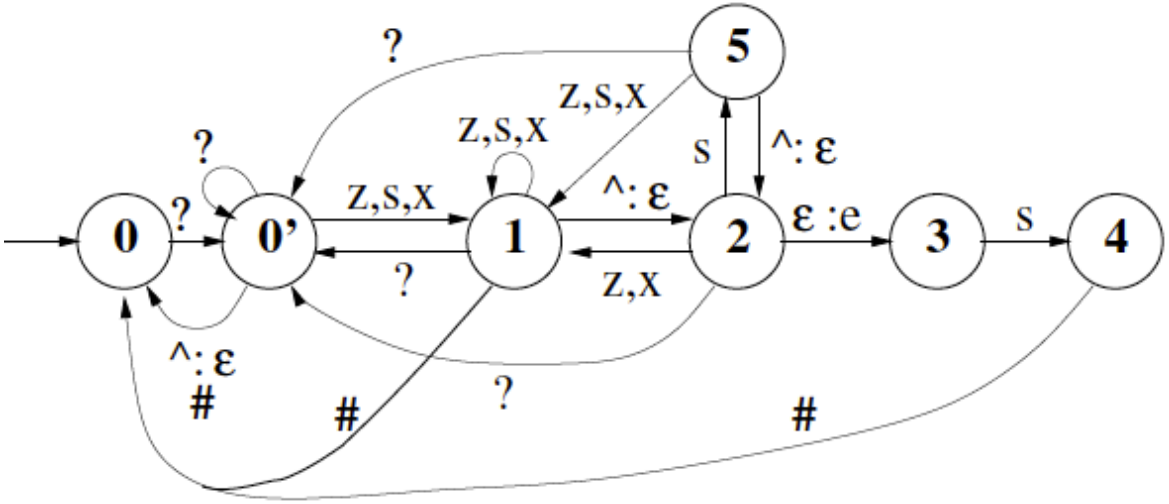


Quick Exercise



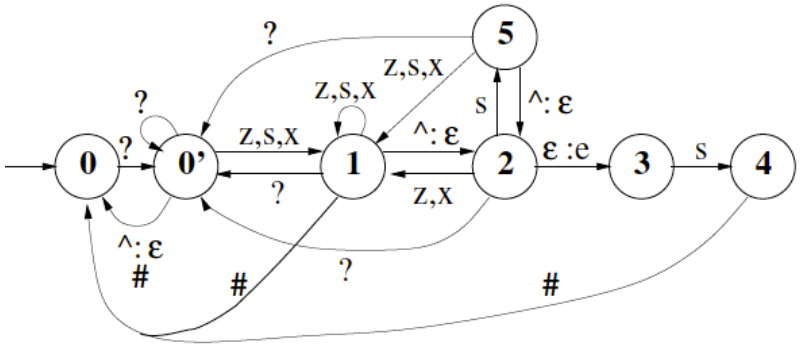
- Apply this backwards to translate from surface to intermediate form.
- Starting from state 0, how many sequences of transitions are compatible with the input string `asses' ?

Quick Exercise



- $0 \xrightarrow{a} 0' \xrightarrow{s} 1 \xrightarrow{s} 1 \xrightarrow{\epsilon} 2 \xrightarrow{e} 3 \xrightarrow{s} 4$, output ass^s

Quick Exercise



On the input string 'asses', 10 transition sequences are possible!

- $0 \xrightarrow{a} 0' \xrightarrow{s} 1 \xrightarrow{s} 1 \xrightarrow{\epsilon} 2 \xrightarrow{e} 3 \xrightarrow{s} 4$, output ass^s
- $0 \xrightarrow{a} 0' \xrightarrow{s} 1 \xrightarrow{s} 1 \xrightarrow{\epsilon} 2 \xrightarrow{e} 0' \xrightarrow{s} 1$, output ass^es
- $0 \xrightarrow{a} 0' \xrightarrow{s} 1 \xrightarrow{s} 1 \xrightarrow{e} 0' \xrightarrow{s} 1$, output asses
- $0 \xrightarrow{a} 0' \xrightarrow{s} 1 \xrightarrow{\epsilon} 2 \xrightarrow{s} 5 \xrightarrow{\epsilon} 2 \xrightarrow{e} 3 \xrightarrow{s} 4$, output as^s^s
- $0 \xrightarrow{a} 0' \xrightarrow{s} 1 \xrightarrow{\epsilon} 2 \xrightarrow{s} 5 \xrightarrow{\epsilon} 2 \xrightarrow{e} 0' \xrightarrow{s} 1$, output as^s^es
- $0 \xrightarrow{a} 0' \xrightarrow{s} 1 \xrightarrow{\epsilon} 2 \xrightarrow{s} 5 \xrightarrow{e} 0' \xrightarrow{s} 1$, output as^ses
- Four of these can also be followed by $1 \xrightarrow{\epsilon} 2$ (output ^).

Hidden Markov Model

Markov Chain

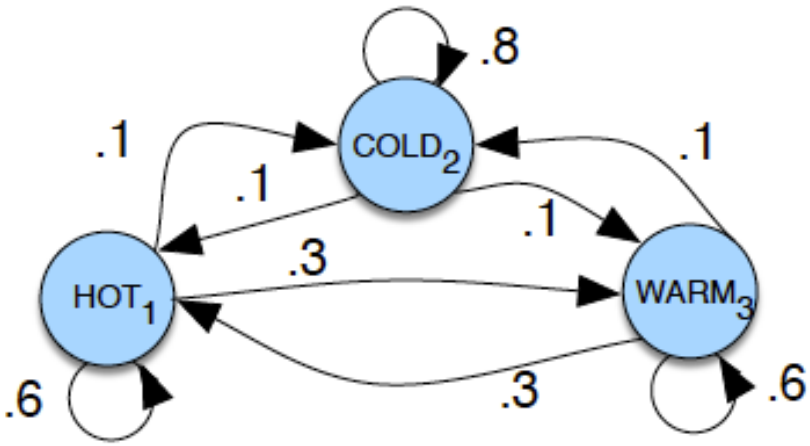
- Tells us about the probabilities of sequences.
- words, tags, symbols,…..

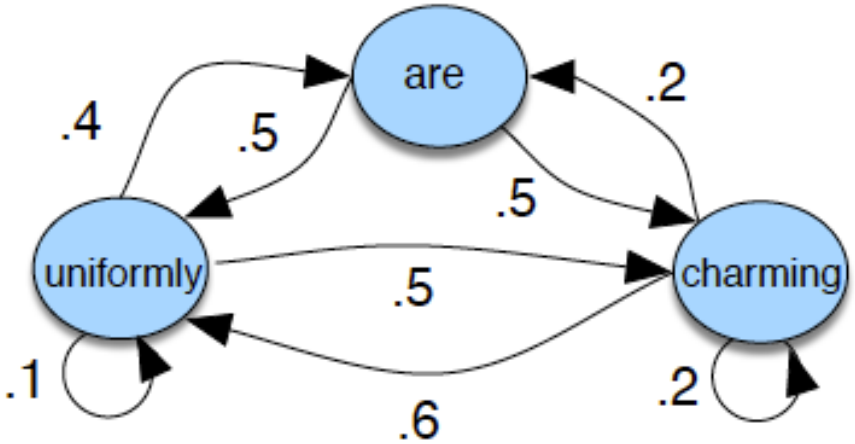
- Property
  - predict the future in the sequence based on the current state.
  - The states before the current state have no impact on the future except via the current state.

Markov Assumption: $P(q_i|q_1...q_{i-1}) = P(q_i|q_{i-1})$

- Example
  - Predict tomorrow's weather from today's weather
  - Not allowed to look at yesterday's weather.

Markov Chain- Examples



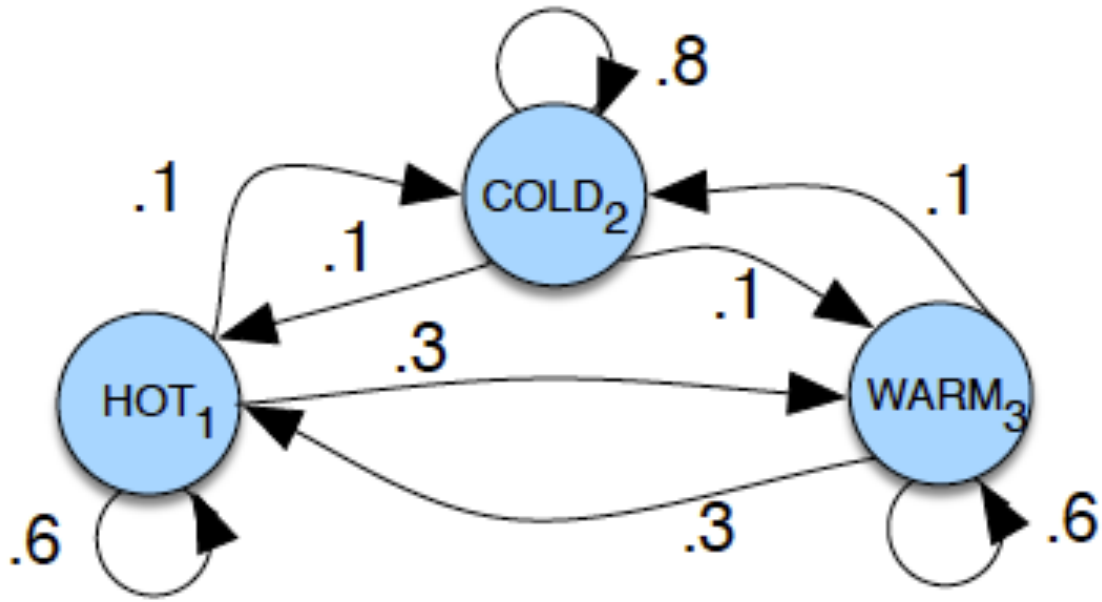


Markov Chain - Formal Definition

$Q = q_1 q_2 \ldots q_N$	a set of N **states**
$A = a_{11} a_{12} \ldots a_{N1} \ldots a_{NN}$	a **transition probability matrix** A, each a_{ij} representing the probability of moving from state i to state j, s.t. $\sum_{j=1}^{n} a_{ij} = 1 \quad \forall i$
$\pi = \pi_1, \pi_2, \ldots, \pi_N$	an **initial probability distribution** over states. π_i is the probability that the Markov chain will start in state i. Some states j may have $\pi_j = 0$, meaning that they cannot be initial states. Also, $\sum_{i=1}^{N} \pi_i = 1$

Quick Exercise

- π = [.1, .7, .2]
- compute the probability of each of the following sequences:
  - hot hot hot hot
  - cold hot cold hot
  - warm hot cold warm



Hidden Markov Model

- Markov chain
  - probability for a sequence of observable events
- Hidden events
  - Not explicit
  - We don’t observe them directly
- Example
  - We don’t normally observe part-of-speech tags in a text
  - Speech Recognition

Hidden Markov Model

- Example (Eisner task)
  - a task proposed by Jason Eisner (2002)
  - Imagine that you are a climatologist in the year 2799 studying the history of global warming for one city, say Baltimore.
  - You cannot find any records of the weather in Baltimore for the summer of 2025
  - But you do find Jason Eisner's diary, which lists how many ice creams Jason ate every day that summer.
  - Our goal is to use these observations to estimate the temperature every day.
  - We'll simplify this weather task by assuming there are only two kinds of days: cold (C) and hot (H).

Hidden Markov Model

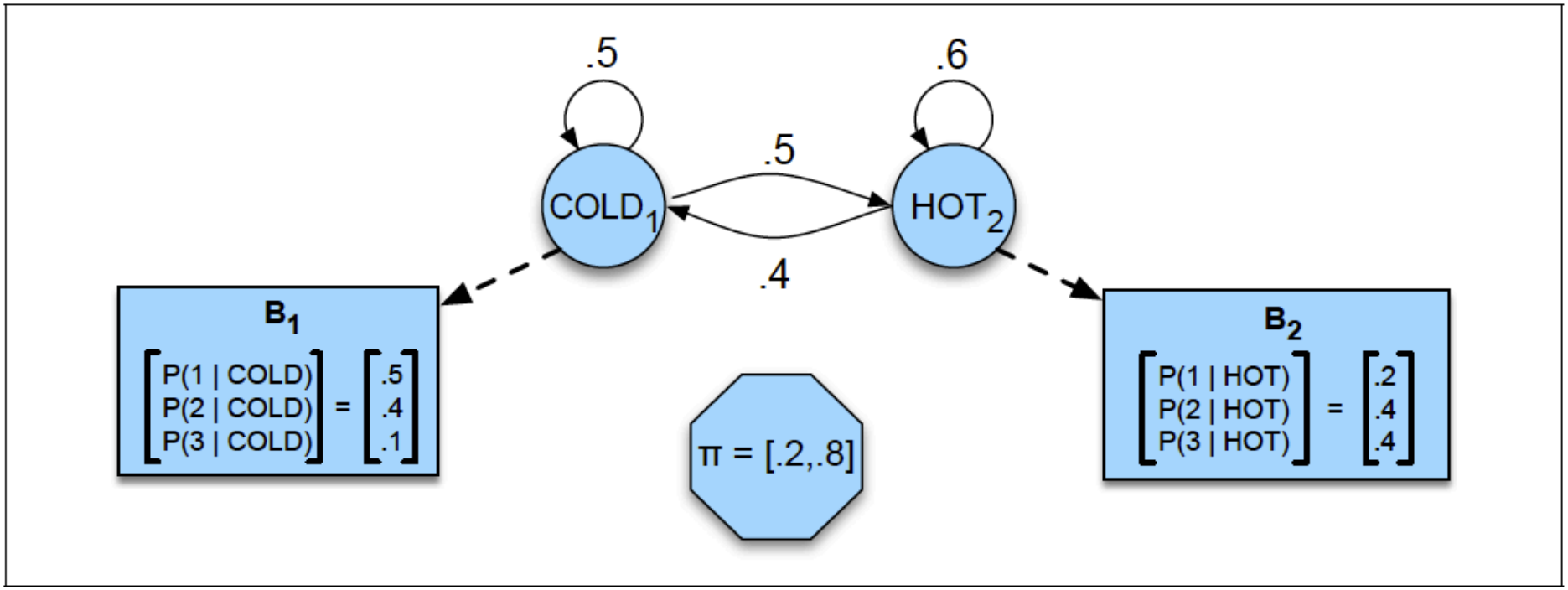
- Example (Eisner task)

✓ *Given a sequence of observations O (each an integer representing the number of ice creams eaten on a given day)*

✓ *Find the 'hidden' sequence Q of weather states (Hot or Cold) which caused Jason to eat the ice cream.*

Hidden Markov Model

- Example (Eisner task)



Hidden Markov Model

$Q = q_1 q_2 \ldots q_N$	a set of N **states**
$A = a_{11} \ldots a_{ij} \ldots a_{NN}$	a **transition probability matrix** A, each a_{ij} representing the probability of moving from state i to state j, s.t. $\sum_{j=1}^{N} a_{ij} = 1 \quad \forall i$
$B = b_i(o_t)$	a sequence of **observation likelihoods**, also called **emission probabilities**, each expressing the probability of an observation o_t (drawn from a vocabulary $V = v_1, v_2, \ldots, v_V$) being generated from a state q_i
$\pi = \pi_1, \pi_2, \ldots, \pi_N$	an **initial probability distribution** over states. π_i is the probability that the Markov chain will start in state i. Some states j may have $\pi_j = 0$, meaning that they cannot be initial states. Also, $\sum_{i=1}^{n} \pi_i = 1$

Hidden Markov Model

A first-order hidden Markov model follows two assumptions.

$$P(q_i|q_1 \ldots q_{i-1}) = P(q_i|q_{i-1})$$

$$P(o_i|q_1 \ldots q_i, \ldots, q_T, o_1, \ldots, o_i, \ldots, o_T) = P(o_i|q_i)$$

Hidden Markov Model

Three Fundamental Tasks

Problem 1 (Likelihood):	Given an HMM $\lambda = (A, B)$ and an observation sequence O, determine the likelihood $P(O\|\lambda)$.
Problem 2 (Decoding):	Given an observation sequence O and an HMM $\lambda = (A, B)$, discover the best hidden state sequence Q.
Problem 3 (Learning):	Given an observation sequence O and the set of states in the HMM, learn the HMM parameters A and B.

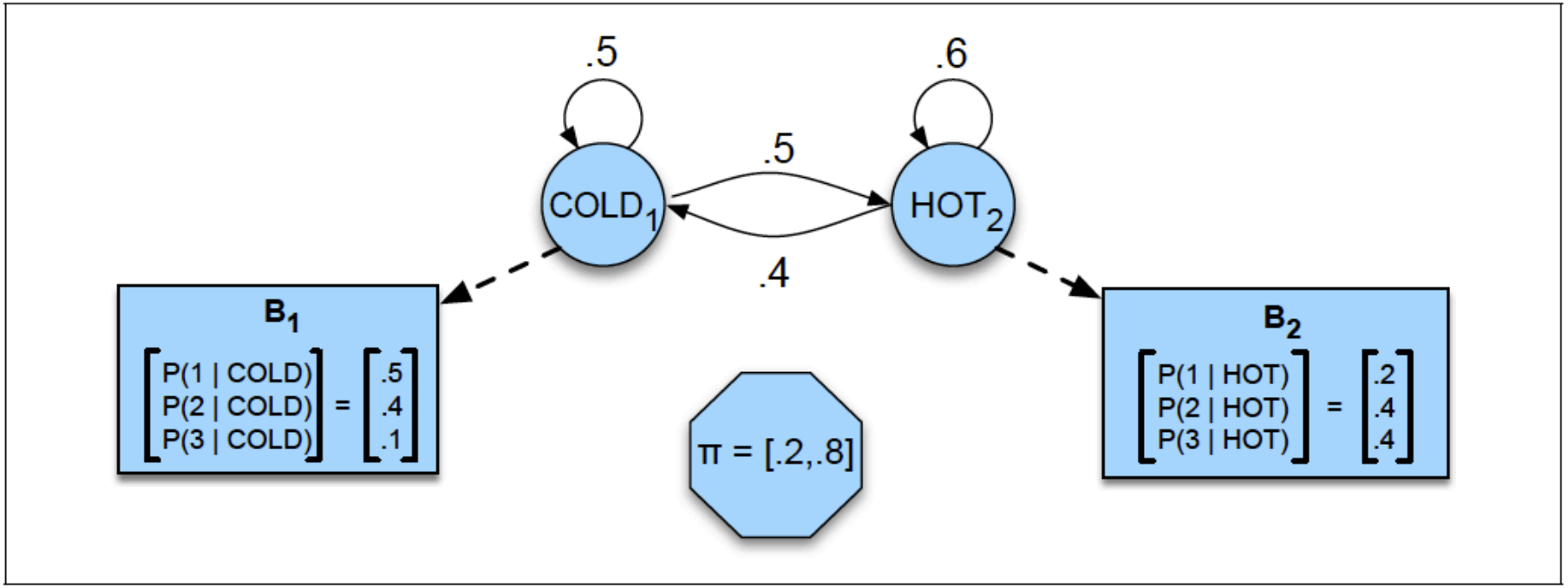
Likelihood Computation

- Computing Likelihood: Given an HMM λ = (A, B) and an observation sequence O, determine the likelihood P(OI λ)

- We want to determine the probability of an ice cream observation sequence like 3 1 3, but we don't know what the hidden state sequence is!

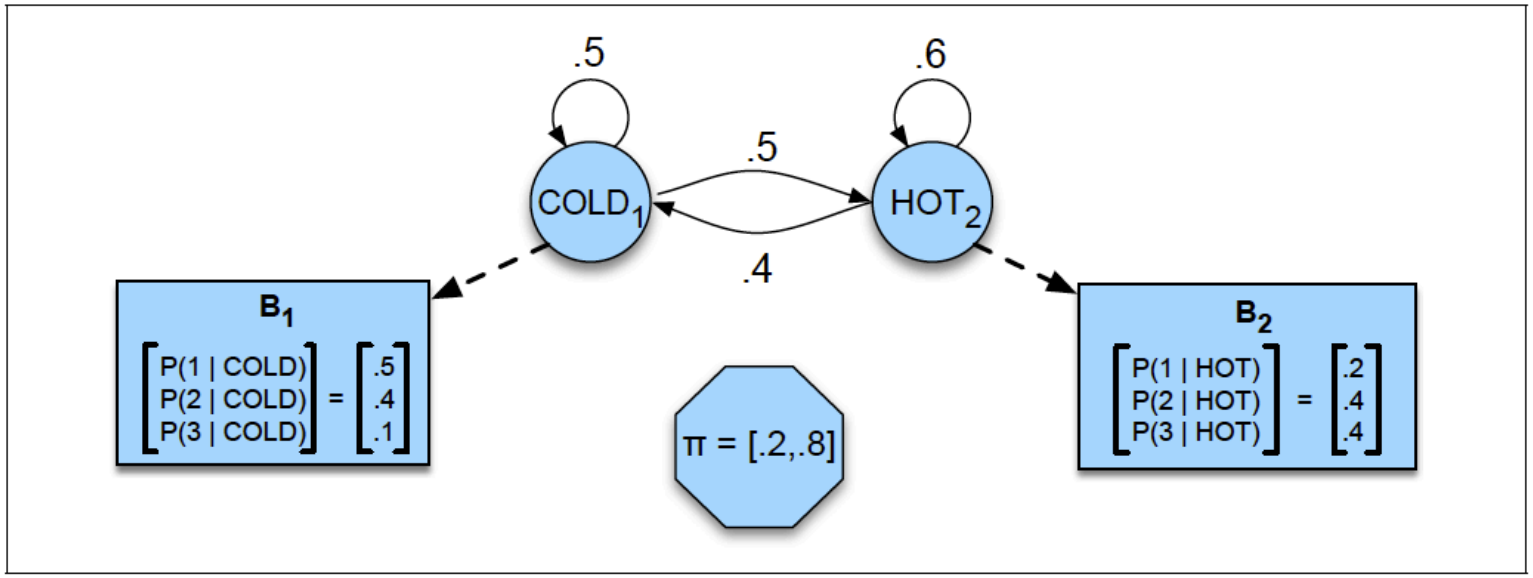
Likelihood Computation

- Let's make it simple
  - For a given hidden state sequence hot hot cold, Can we compute the output likelihood of 3 1 3 ?



Likelihood Computation

- Let's make it simple
  - For a given hidden state sequence hot hot cold, Can we compute the output likelihood of 3 1 3 ?



*P(3 1 3, hot hot cold) = P(hot | start) * P(hotlhot) *P(cold | hot)*P(3 | hot)*P(1 | hot)* P(3lcold)*

Likelihood Computation

- Joint probability of being in a particular weather sequence Q and generating a particular sequence O of ice-cream events.

$$P(O,Q) = P(O|Q) \times P(Q) = \prod_{i=1}^{T} P(o_i|q_i) \times \prod_{i=1}^{T} P(q_i|q_{i-1})$$

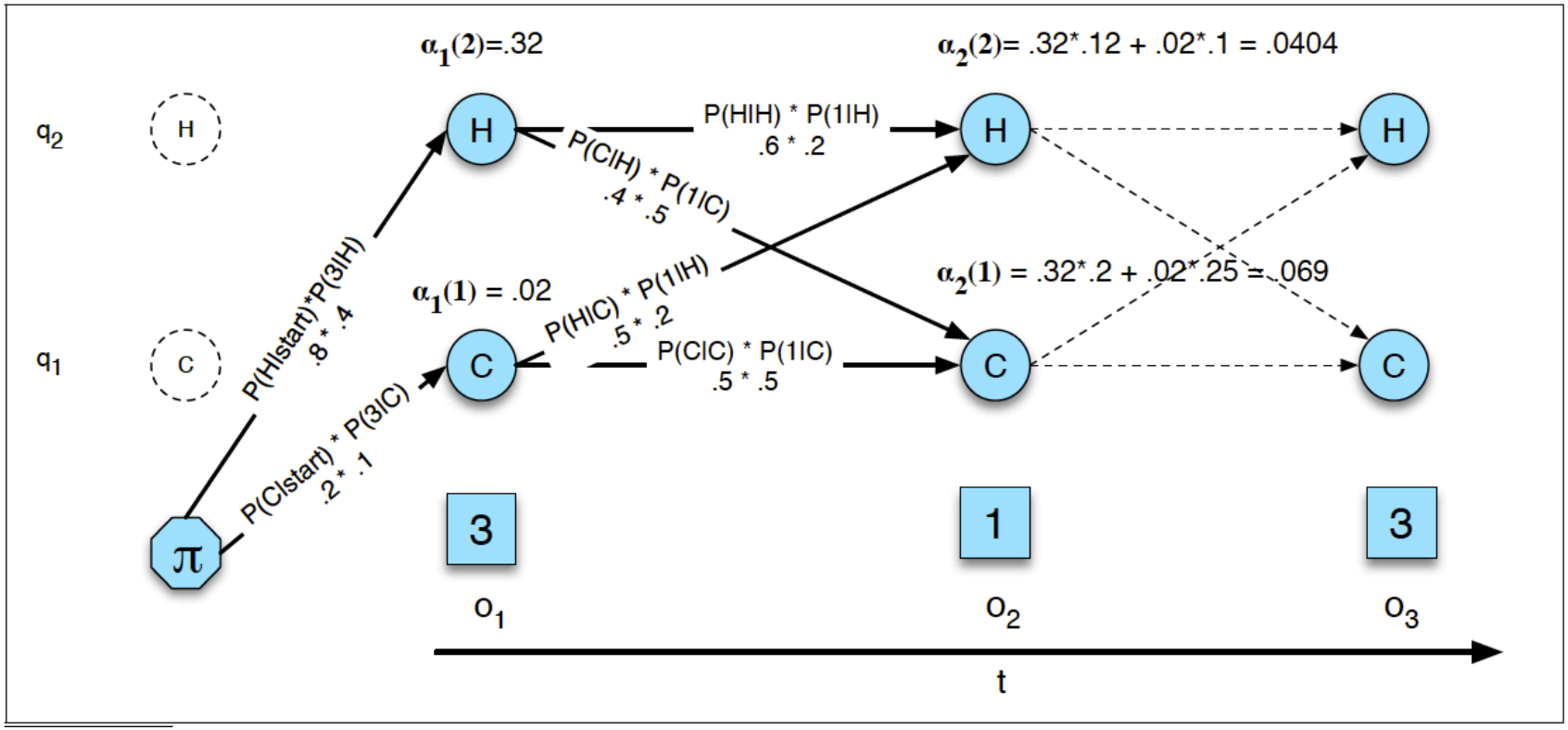
$$P(O) = \sum_{Q} P(O,Q) = \sum_{Q} P(O|Q)P(Q)$$

For an HMM with N hidden states and an observation sequence of T observations, there are N^T possible hidden sequences.

Likelihood Computation: The Forward Algorithm

- Kind of dynamic programming algorithm
  - uses a table to store intermediate values as it builds up the probability of the observation sequence.
- Computes the observation probability by summing over the probabilities of all possible hidden state paths that could generate the observation sequence

Likelihood Computation: The Forward Algorithm



Hidden states are in circles, observations in squares

α_t (j) represents the probability of being in state j after seeing the first t observations, given the HMM λ

Likelihood Computation: The Forward Algorithm

$$\alpha_t(j) = \sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij} b_j(o_t)$$

$\alpha_{t-1}(i)$	the **previous forward path probability** from the previous time step
a_{ij}	the **transition probability** from previous state q_i to current state q_j
$b_j(o_t)$	the **state observation likelihood** of the observation symbol o_t given the current state j